

Core Breach

CENG 454, Game Programming, Spring 2025-2026

Homework 3: Pattern-Driven Systems Prototype

Full Name	Alp Doruk Şengün
Student ID	230444401
GitHub URL	https://github.com/AlpoTheo/CENG454-HW3-AlpDoruk-Sengun-230444401
Submission Date	17 May 2026
Unity Version	Unity 6 (URP, 2D)

1. Gameplay Overview

Core Breach is a small 2D top-down prototype. The player must protect a yellow energy core that sits in the middle of the arena. Red enemies appear at the edges of the screen and walk toward the core. The whole game takes place in one short round.

Each wave behaves differently. Wave 1 enemies walk straight to the core. Wave 2 enemies move side to side while they come in, so they are harder to hit. Wave 3 enemies spiral around the core instead of going straight. This is where I use the Strategy pattern. The EnemyController class is the same for every enemy, but the movement behaviour can be changed per wave.

The player moves with WASD, aims with the mouse, and fires with left click. The weapon is built at the start of the scene as a chain of decorators. The base weapon is wrapped by RapidFireDecorator, and that is wrapped by DoubleShotDecorator. Every click fires two bullets at +/-12 degrees, at two times the base fire rate. Projectiles and enemies are taken from object pools, so they are not created with Instantiate during play.

Goal, win condition, lose condition

Goal. Keep the energy core alive through three waves.

Win. All three waves are cleared (no live enemies and no more waves to spawn). The win panel appears and Time.timeScale is set to zero.

Lose. Core HP becomes zero. The lose panel appears and time stops in the same way.

Length. One run takes around 2 to 4 minutes. The exact time depends on how the player keeps enemies away from the core.

2. Architecture Overview

GameSetup is the composition root of the scene. It builds the weapon decorator chain when the scene starts, gives the wrapped IWeapon to PlayerShooter, and then calls WaveManager.Init. After this point, most of the communication between systems happens through C# events instead of direct references. I also kept GameManager small. It only listens to events and shows the win or lose panel.

Folder map

Assets/Scripts/

Interfaces/	IDamageable, IMovementStrategy, IWeapon, IPoolable
Core/	EnergyCore (Observer publisher)
Player/	PlayerController, PlayerShooter
Enemy/	EnemyController (uses IMovementStrategy)
Strategies/	DirectMoveStrategy, ZigZagMoveStrategy, OrbitMoveStrategy
Weapons/	BaseWeapon, WeaponDecorator, RapidFireDecorator, DoubleShotDecorator
Pool/	ObjectPool, Projectile
Managers/	GameSetup, WaveManager, GameManager
UI/	HUDController

How systems talk

EnergyCore is the publisher for damage events. TakeDamage(amount) raises OnHealthChanged(current, max) on every hit. When health reaches zero, it raises OnCoreDestroyed one time. HUDController subscribes to both events. The first event updates the HP slider. The second event hides the HUD when the run ends. GameManager also subscribes to OnCoreDestroyed and shows the lose panel. The two subscribers do not know about each other, and EnergyCore does not know about either of them.

WaveManager publishes OnWaveStarted(int) and OnAllWavesCleared. HUDController updates the wave label from the first event. GameManager shows the win panel from the second event. Inside WaveManager I use a small Phase enum state machine with four states (WaitingToStart, WaveActive, BetweenWaves, Finished) that runs in Update. EnemyController has a static OnEnemyDied event, so the wave manager can decrement its alive counter without keeping a list of enemy references.

Player input uses the new **UnityEngine.InputSystem**: Keyboard.current and Mouse.current with null guards on both. The player movement runs in FixedUpdate on a Rigidbody2D. The aim rotation and weapon firing run in Update. Projectiles and enemies are both pooled, and they reset their own state through IPoolable.OnSpawn and OnDestroy.

3. Pattern Justification Table

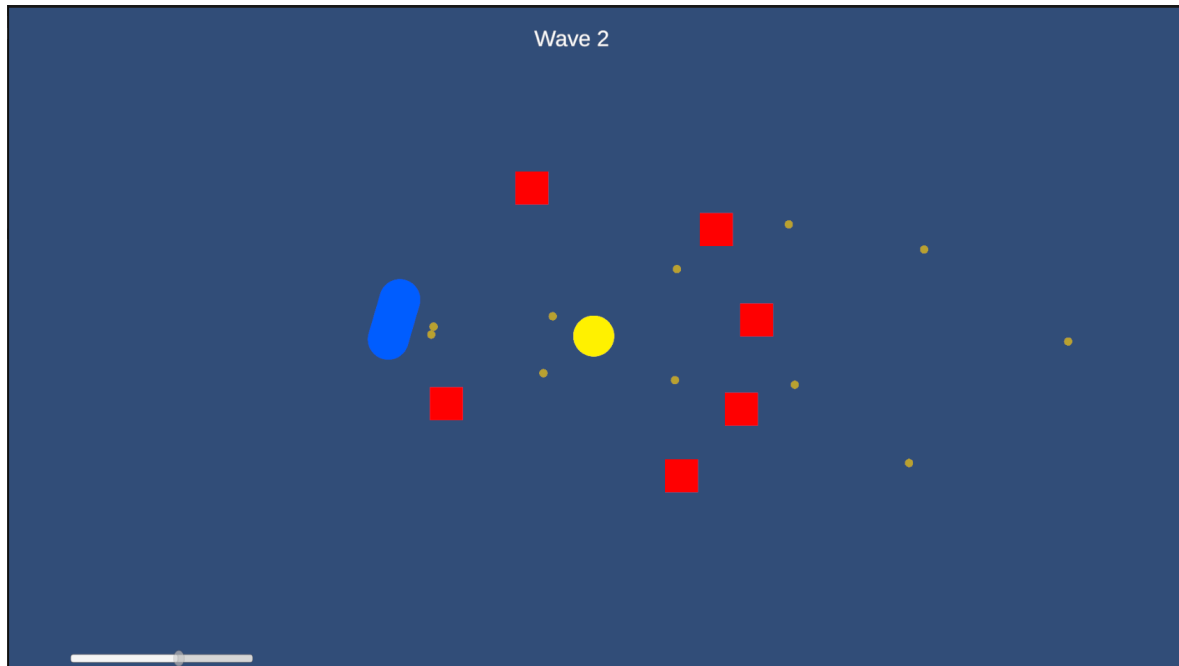
Each row in the table points to specific files in the project. Every pattern solves a real problem in the design. If I removed any one of them, the code would need many type checks, or it would create performance issues from Instantiate calls during play.

Pattern	Classes / Interfaces Involved	Problem Solved	Why This Choice Was Appropriate
Observer	EnergyCore.OnHealthChanged, EnergyCore.OnCoreDestroyed, WaveManager.OnWaveStarted, WaveManager.OnAllWavesCleared, EnemyController.OnEnemyDied. Subscribers: HUDController, GameManager, WaveManager.	One change in the world (core damaged, wave done, enemy died) needs to drive several unrelated reactions: HUD update, win or lose panel, kill counter. A direct call chain would couple all of them to EnergyCore.	C# events keep the publisher ignorant of who listens. I can drop in an AudioManager later that subscribes to OnHealthChanged and EnergyCore does not need to know about it.
Strategy	IMovementStrategy + DirectMoveStrategy, ZigZagMoveStrategy, OrbitMoveStrategy. Owner: EnemyController. Selector: WaveManager.SelectStrategyForWave.	Different waves should feel different (straight rush, weaving, spiral). Hard-coding 'if wave == 2 zigzag' inside EnemyController would tie the enemy class to wave-tuning logic.	EnemyController only depends on the IMovementStrategy contract. Adding a new movement style is one new file plus one switch case. No edits to the enemy, no edits to existing strategies.
Object Pool	ObjectPool (generic) + IPoolable. Pooled types: Projectile, EnemyController.	Projectiles fire several times per second and enemies arrive in bursts of 5 to 9. Calling Instantiate / Destroy on every shot would produce GC spikes.	Reusing GameObjects avoids allocation churn. IPoolable.OnSpawn and OnDespawn give each pooled type a clean place to reset health, timers, and velocity before reuse.
Decorator	IWeapon + BaseWeapon + WeaponDecorator (abstract) + RapidFireDecorator + DoubleShotDecorator. Composed in GameSetup.	Weapon mods (rapid fire, multi-shot, future ones like piercing) should stack at runtime instead of exploding into a subclass per combination.	Each decorator wraps an IWeapon and only overrides what it changes (FireRate or Fire). A subclass-per-combo approach would need Base, Rapid, Double, RapidDouble, etc. Decorator gives N choose K for free.
Interfaces	IDamageable (EnergyCore, EnemyController), IMovementStrategy (3 strategies), IWeapon (BaseWeapon + decorators), IPoolable (Projectile, EnemyController).	Removes direct concrete dependencies between systems. Projectile damages 'whatever IDamageable it hits' and never branches on enemy versus core.	Every interface has at least two implementations and crosses a system boundary, which is the seam test the brief is asking for.
Singleton	Not used.	—	I considered making GameManager a Singleton but every other system already talks to it through events. There is no caller that needs a global handle. Keeping it a plain MonoBehaviour avoids the lifecycle and cross-scene headaches Singletons usually drag along.

What changed during the build. I started with only two movement strategies (Direct and ZigZag). After I played Wave 3 a few times, the movement felt repetitive. So I added OrbitMoveStrategy as a third option. This was one new file plus one extra case in WaveManager.SelectStrategyForWave, with no edits in other files. This is the kind of change the brief wants from a Strategy seam. The order of the decorator chain also changed in the middle of the project. My first version wrapped BaseWeapon with DoubleShot, and I added RapidFire on top later. But RapidFire feels better when DoubleShot inherits the higher fire rate, so I rewrote the chain in GameSetup so that RapidFire is applied first.

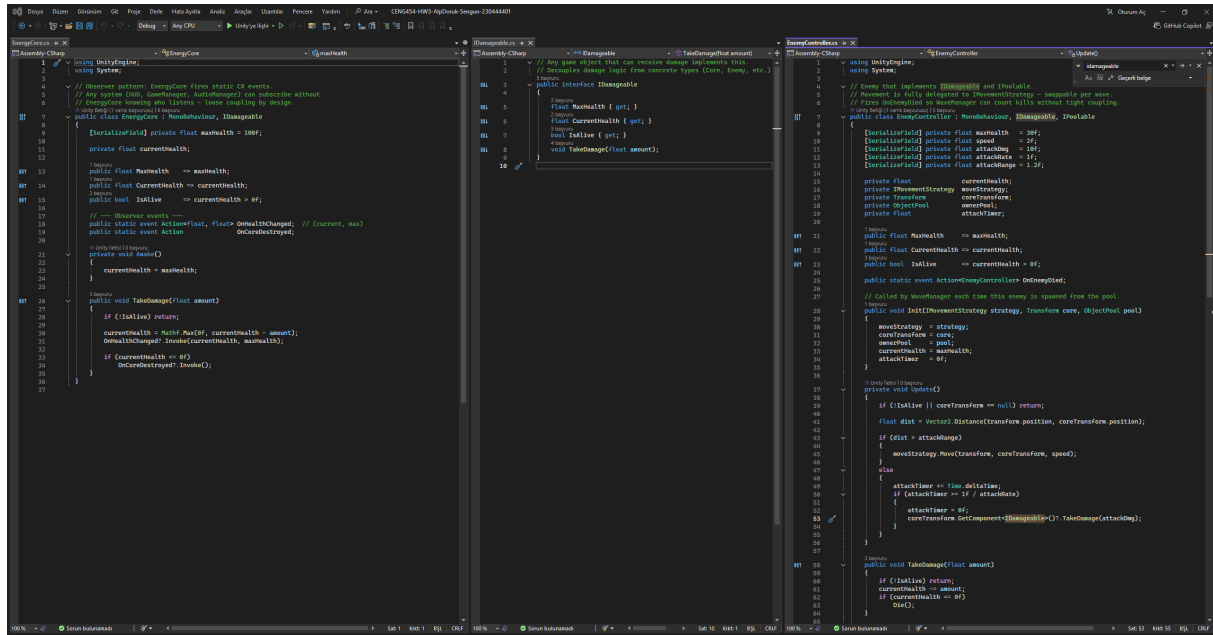
4. Screenshots and Evidence

Screenshot 1: Playable arena and the defendable objective



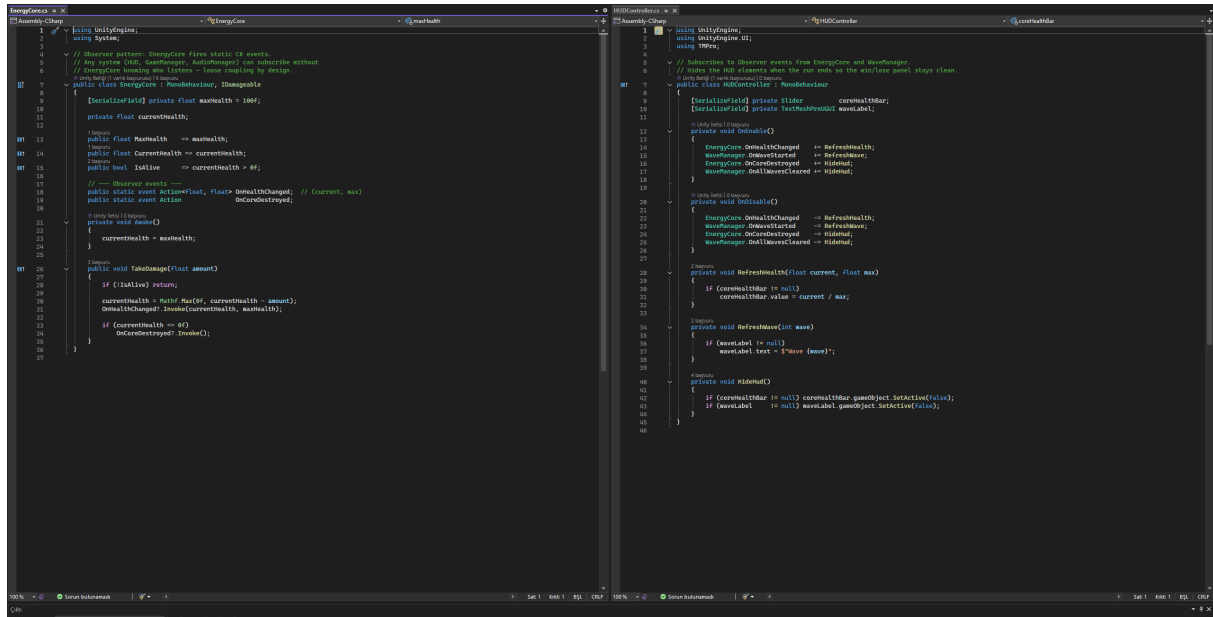
Wave 2 in progress. Yellow circle is the EnergyCore (the objective). Blue capsule is the player. Red squares are enemies, the small yellow dots are active projectiles fetched from the pool. The white slider on the bottom-left is the core HP, driven by `EnergyCore.OnHealthChanged`.

Screenshot 2: Custom interface with two implementations



IDamageable contract on the left. EnergyCore (core HP) and EnemyController (enemy HP) both implement it. Projectile.OnTriggerEnter2D talks to the interface, never to the concrete classes, so a future destructible barrier can be hit by the same code without any change.

Screenshot 3: Observer-style event flow



Left: EnergyCore raises OnHealthChanged and OnCoreDestroyed inside TakeDamage. Right: HUDController subscribes in OnEnable and unsubscribes in OnDisable for both EnergyCore events plus WaveManager events. GameManager subscribes to the same OnCoreDestroyed without HUDController knowing about it. That is the decoupling the pattern is bought for.

[illegible]

IMovementStrategy.Move(sell, target, speed) is the only method on the contract. DirectMoveStrategy walks straight, ZigZagMoveStrategy weaves with sin, and OrbitMoveStrategy (see 4b) spirals inward. EnemyController stores an IMovementStrategy field and calls strategy.Move() in Update. There is no type switch.

Screenshot 4b: Strategy per-wave selection

```
// IMovementStrategy.cs
1 using UnityEngine;
2
3 // Strategy pattern contract for enemy movement
4 // EnemyController depends on this interface, not on concrete wave classes.
5 public interface IMovementStrategy
6 {
7     void Move(Transform self, Transform target, float speed);
8 }
9
// DirectMovementStrategy.cs
1 using UnityEngine;
2
3 // Strategy 1: Enemy moves in a straight line toward the core
4 // Used in wave 1 - simple and predictable
5 public class DirectMovementStrategy : IMovementStrategy
6 {
7     public void Move(Transform self, Transform target, float speed)
8     {
9         Vector3 direction = (target.position - self.position).normalized;
10        self.position += direction * speed * Time.deltaTime;
11    }
12 }
13
// ZigZagMovementStrategy.cs
1 using UnityEngine;
2
3 // Strategy 2: Enemy moves in a zigzag pattern around the core
4 // Used in wave 2 - more complex and unpredictable
5 public class ZigZagMovementStrategy : IMovementStrategy
6 {
7     public void Move(Transform self, Transform target, float speed)
8     {
9         // Implementation of zigzag movement
10    }
11 }
12
// WaveManager.cs
1 using UnityEngine;
2
3 // WaveManager
4 // Controls the flow of waves and enemy spawning
5 public class WaveManager : MonoBehaviour
6 {
7     // Wave types
8     public enum WaveType
9     {
10        Direct,
11        ZigZag,
12        Orbit
13    }
14
15     // Wave data
16     public WaveData[] waves;
17
18     // Current wave
19     private int currentWave = 0;
20
21     // Current phase
22     private Phase currentPhase = Phase.None;
23
24     // Timer for wave transitions
25     private float timer = 0f;
26
27     // Enemy pool
28     private EnemyPool enemyPool;
29
30     // Core transform
31     private Transform coreTransform;
32
33     // Start method
34     void Start()
35     {
36        // Initialize enemy pool
37        enemyPool = GetComponent<EnemyPool>().Init();
38
39        // Set core transform
40        coreTransform = GetComponent<Core>().transform;
41    }
42
43     // Update method
44     void Update()
45     {
46        // Check if current wave is finished
47        if (currentWave == waves.Length - 1)
48        {
49            // All waves finished, invoke OnWaveFinished
50            OnWaveFinished();
51            return;
52        }
53
54        // Check if current phase is finished
55        if (currentPhase == Phase.None)
56        {
57            // No phase, start with Direct
58            currentPhase = Phase.Direct;
59            timer = 0f;
60            return;
61        }
62
63        // Check if current phase is finished
64        if (currentPhase == Phase.Direct)
65        {
66            // Direct phase finished, start with ZigZag
67            currentPhase = Phase.ZigZag;
68            timer = 0f;
69            return;
70        }
71
72        // Check if current phase is finished
73        if (currentPhase == Phase.ZigZag)
74        {
75            // ZigZag phase finished, start with Orbit
76            currentPhase = Phase.Orbit;
77            timer = 0f;
78            return;
79        }
80
81        // Check if current phase is finished
82        if (currentPhase == Phase.Orbit)
83        {
84            // Orbit phase finished, start with Direct
85            currentPhase = Phase.Direct;
86            timer = 0f;
87            return;
88        }
89
90        // Check if timer is zero
91        if (timer == 0f)
92        {
93            // Start new wave
94            StartWave();
95            return;
96        }
97
98        // Decrease timer
99        timer -= Time.deltaTime;
100    }
101
102     // StartWave method
103     void StartWave()
104     {
105        // Get current wave data
106        WaveData waveData = waves[currentWave];
107
108        // Set current wave
109        currentWave++;
110
111        // Set current phase
112        currentPhase = waveData.phase;
113
114        // Set timer
115        timer = waveData.timer;
116
117        // Spawn enemies
118        for (int i = 0; i < waveData.enemiesLive; i++)
119        {
120            SpawnEnemy();
121        }
122
123        // Set phase
124        currentPhase = Phase.Active;
125    }
126
127     // SpawnEnemy method
128     void SpawnEnemy()
129     {
130        // Get random angle
131        float angle = UnityEngine.Random.Range(0f, 360f) * Mathf.Deg2Rad;
132
133        // Get random position
134        Vector3 spawnPos = new Vector3(Mathf.Cos(angle), Mathf.Sin(angle)) * spawnRadius;
135
136        // Create enemy object
137        GameObject obj = enemyPool.Get();
138
139        // Set transform position
140        obj.transform.position = spawnPos;
141
142        // Set movement strategy
143        IMovementStrategy strategy = SelectStrategyForWave(currentWave);
144
145        // Get component
146        obj.GetComponent<EnemyController>().Init(strategy, coreTransform, enemyPool);
147    }
148
149     // OnWaveFinished method
150     void OnWaveFinished()
151     {
152        // Set enemies alive
153        enemiesAlive = Mathf.Max(0, enemiesAlive - 1);
154    }
155
156     // SelectStrategyForWave method
157     static IMovementStrategy SelectStrategyForWave(int wave)
158     {
159        switch (wave)
160        {
161            case 1: return new DirectMovementStrategy();
162            case 2: return new ZigZagMovementStrategy();
163            default: return new OrbitMovementStrategy();
164        }
165    }
166 }
```

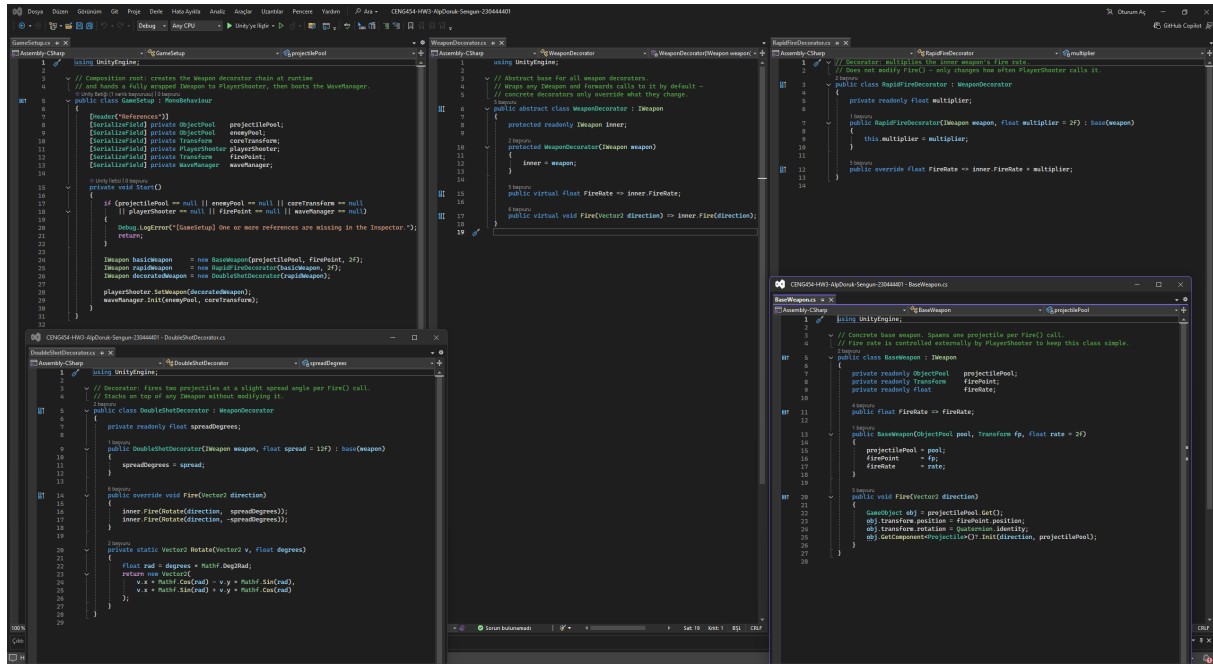
WaveManager.SpawnEnemy calls SelectStrategyForWave(currentWave) and injects the result into EnemyController.Init. Wave 1 is Direct, Wave 2 is ZigZag, Wave 3+ is Orbit. Adding a fourth strategy would only touch this one switch.

Screenshot 5: Object Pool runtime evidence



The Hierarchy panel shows the ProjectilePool and EnemyPool objects with all their pre-instantiated children parked under them. Inactive instances live under the pool transform; on Get() they get activated and reparented into the world; on ReturnToPool they get deactivated and re-parented back. The Inspector panel is showing a live-running scene during Wave 1.

Screenshot 6a: Decorator (IWeapon and chain classes)



IWeapon is the contract. BaseWeapon is the concrete leaf that talks to the ObjectPool. WeaponDecorator is an abstract base that holds an inner IWeapon and forwards by default. RapidFireDecorator overrides FireRate only. DoubleShotDecorator overrides Fire only and uses a small rotation helper to fan two projectiles at +/-12 degrees.

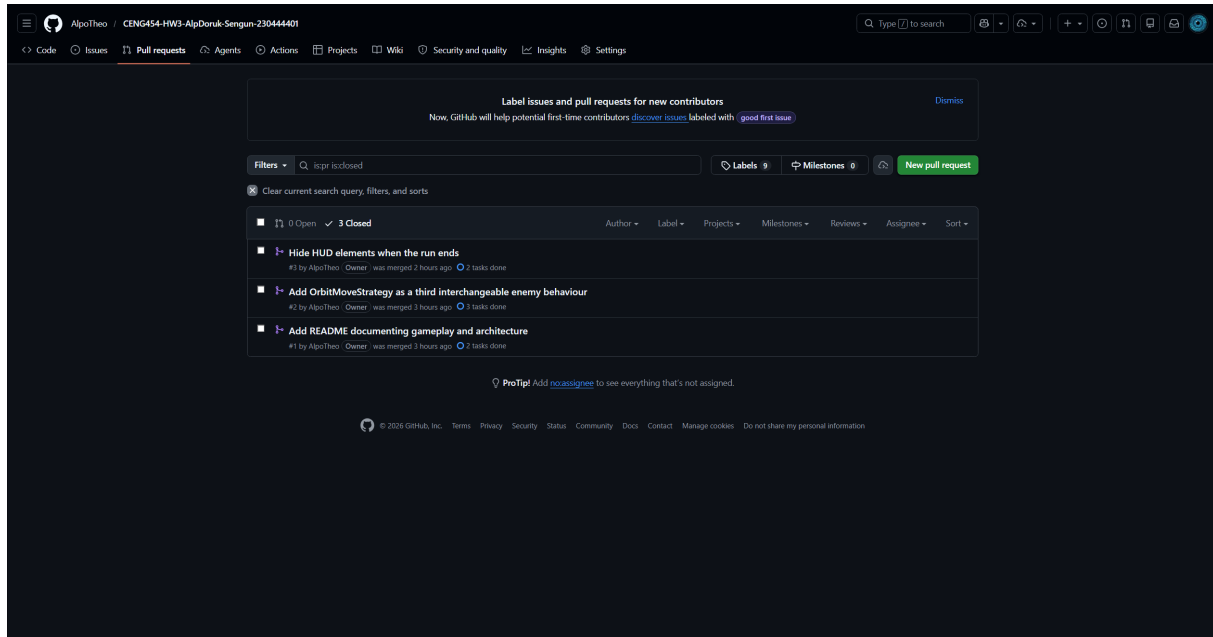
Screenshot 6b: Decorator composition root

```
GameSetup.cs
Assembly-CSharp
GameSetup
Start()

1 using UnityEngine;
2
3 // Composition root: creates the Weapon decorator chain at runtime
4 // and hands a fully wrapped IWeapon to PlayerShooter, then boots the WaveManager.
5 public class GameSetup : MonoBehaviour
6 {
7     [Header("References")]
8     [SerializeField] private ObjectPool projectilePool;
9     [SerializeField] private ObjectPool enemyPool;
10    [SerializeField] private Transform coreTransform;
11    [SerializeField] private PlayerShooter playerShooter;
12    [SerializeField] private Transform firePoint;
13    [SerializeField] private WaveManager waveManager;
14
15    private void Start()
16    {
17        if (projectilePool == null || enemyPool == null || coreTransform == null
18            || playerShooter == null || firePoint == null || waveManager == null)
19        {
20            Debug.LogError("[GameSetup] One or more references are missing in the Inspector.");
21            return;
22        }
23
24        IWeapon basicWeapon = new BaseWeapon(projectilePool, firePoint, 2f);
25        IWeapon rapidWeapon = new RapidFireDecorator(basicWeapon, 2f);
26        IWeapon decoratedWeapon = new DoubleShotDecorator(rapidWeapon);
27
28        playerShooter.SetWeapon(decoratedWeapon);
29        waveManager.Init(enemyPool, coreTransform);
30    }
31
32 }
```

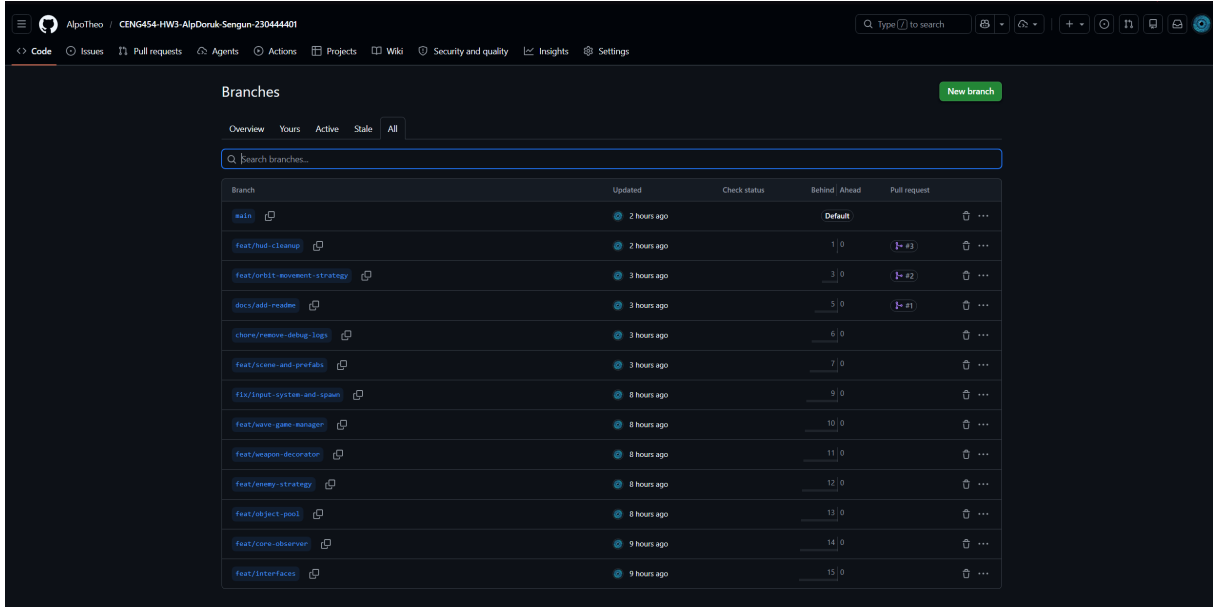
GameSetup.Start composes the chain in three lines: BaseWeapon, then RapidFireDecorator(2x), then DoubleShotDecorator. The fully wrapped IWeapon is handed to PlayerShooter via SetWeapon. PlayerShooter holds an IWeapon, never a concrete type, so the chain can be reordered or extended without touching the shooter.

Screenshot 7a: Repository merged pull requests



Three closed pull requests on main: docs/add-readme (#1), feat/orbit-movement-strategy (#2), feat/hud-cleanup (#3). Each PR was opened from a feature branch and merged after testing the change in Play mode. This satisfies the at-least-two-merged-PRs minimum with one to spare.

Screenshot 7b: Repository feature branches



Branch	Updated	Check status	Behind	Ahead	Pull request
main	2 hours ago			Default	
feat/hud-cleanup	2 hours ago		1	0	#3
feat/orbit-movement-strategy	3 hours ago		3	0	#2
docs/add-readme	3 hours ago		5	0	#1
chore/remove-debug-logs	3 hours ago		6	0	
feat/scene-and-prefabs	3 hours ago		7	0	
fix/input-system-and-spawn	8 hours ago		9	0	
feat/wave-game-manager	8 hours ago		10	0	
feat/weapon-decorator	8 hours ago		11	0	
feat/enemy-strategy	8 hours ago		12	0	
feat/object-pool	8 hours ago		13	0	
feat/core-observer	9 hours ago		14	0	
feat/interfaces	9 hours ago		15	0	

Branches list. Beside main there are 12 short-lived feature branches: feat/interfaces, feat/core-observer, feat/object-pool, feat/enemy-strategy, feat/weapon-decorator, feat/wave-game-manager, fix/input-system-and-spawn, feat/scene-and-prefabs, chore/remove-debug-logs, docs/add-readme, feat/orbit-movement-strategy, feat/hud-cleanup. Each one carries one logical slice of work, so the history is reviewable commit by commit instead of as a single dump.

5. Debugging Story

Bug. WaveManager would not start the first wave, so the game just sat there once I pressed Play.

Setup

I had just finished the first version of WaveManager and wired all the Inspector references in GameSetup. Pressed Play. The scene came up. The player worked, the HUD slider was visible. The wave label stayed empty. No enemies showed up. The whole thing just hung in a 'nothing is happening' way that is much worse than a real exception, because there is no stack trace to start from.

Symptoms observed

The Console printed the [GameSetup] Start log, which proved that GameSetup.Start() ran and called WaveManager.Init() successfully. The [WaveManager] StartWave log that should have appeared after the two-second start delay never showed up. No enemies spawned. No exception was raised. The MonoBehaviour was active and enabled in the Inspector. So the manager was alive, initialised, and then nothing.

Diagnosis

The first version of WaveManager ran the wave logic in a coroutine. Init started a private RunWaves() IEnumerator via StartCoroutine, and the very first line of that coroutine was `yield return new WaitForSeconds(startDelay)`. To narrow the hang down I added two logs, one right before the yield and one right after. The pre-yield log printed. The post-yield log never did, not once.

That meant the coroutine was suspending on the very first WaitForSeconds and never resuming. I tried a couple of small changes (calling StartCoroutine on WaveManager itself rather than from inside Init, switching the wait to `yield return null` as a sanity check, briefly suspecting `Time.timeScale`) and none of them gave me a clean answer. There was no error, no exception, the GameObject was active, the script was enabled. It just did not come back.

At that point I made a call. Rather than spend more time chasing a coroutine that refused to resume on me, rewrite the timing as something I can actually inspect. A coroutine hides its progress inside the IEnumerator state machine, which is exactly the thing I cannot read in the Inspector. An explicit Phase enum, on the other hand, can be a [SerializeField] away from being visible at runtime.

Fix applied

Removed the coroutine entirely. Replaced it with an Update-driven state machine: a private Phase enum with four values (WaitingToStart, WaveActive, BetweenWaves, Finished) plus a single float timer that counts down each frame:

```
private enum Phase { WaitingToStart, WaveActive, BetweenWaves, Finished }
private Phase phase = Phase.WaitingToStart;
private float timer;

private void Update()
{
    switch (phase)
    {
        case Phase.WaitingToStart:
            timer -= Time.deltaTime;
```



```

        if (timer <= 0f) StartNextWave();
        break;
    case Phase.WaveActive:
        if (enemiesAlive <= 0) { /* advance */ }
        break;
    case Phase.BetweenWaves:
        timer -= Time.deltaTime;
        if (timer <= 0f) StartNextWave();
        break;
    }
}

```

Outcome

The enemies spawned correctly on the next Play run. All three waves played in order. The delay between waves was correct. OnAllWavesCleared fired one time after the last enemy in Wave 3 died. I could see the Phase value change in the Inspector during play, so I did not need extra Debug.Log lines to check the state transitions.

What it taught me

Sometimes the best fix is not to repair the broken code, but to use a different approach. A coroutine keeps its state inside the IEnumerator. I cannot see this state from outside. A state machine with an enum is more code, but I can read the value during play. After this bug, I prefer Update plus an enum for any timing logic that is more than a simple delay.

6. Troubleshooting Answer: Debug Report #003 (Ghost Subscriber in a Pool)

The brief gives a hypothetical: a pooled enemy stays subscribed to a core-damaged event, gets recycled, and starts producing duplicate reactions on the next core hit.

(a) Visible symptom

My project has no audio yet, so the symptom would show as a damage problem on the core. The HP slider would drop by more than one enemy should cause. If I add a `Debug.Log` inside the React handler, it prints two times for the first hit, then three times for the next hit, and more after that. Pooled enemies that are inactive are still on the event delegate list. If audio or VFX were added later, the same bug would play duplicate sound effects or stack particle bursts.

(b) Root cause

The enemy class subscribes to `EnergyCore.OnCoreDamaged` inside `Awake`. `Awake` runs only one time, when the `GameObject` is first created. When the enemy dies, the pool calls `SetActive(false)` on it. But making a `GameObject` inactive does not remove a C# event subscription. The static delegate still holds a reference to the inactive enemy. When `OnCoreDamaged` is raised next, every old enemy on the delegate list runs the callback, even the ones that are inactive in the pool.

The bug also gets worse with reuse. When the same enemy is taken from the pool and activated again, any subscribe code that runs again (a second `Awake` call, or an `OnEnable` that subscribes) adds another copy of the subscription. So the same instance can be on the list two or three times. After many reuses, each core hit runs the handler that many times for the same enemy.

(c) Exact fix

Move the subscription out of `Awake`. Subscribe in `OnEnable`, unsubscribe in `OnDisable`. `ObjectPool` already toggles active state on every `Get` and `ReturnToPool`, so `OnDisable` will fire automatically when the enemy is queued back, and `OnEnable` will fire once on reuse:

```
private void OnEnable() { EnergyCore.OnCoreDamaged += React; }  
private void OnDisable() { EnergyCore.OnCoreDamaged -= React; }
```

If subscribing has to happen earlier than `OnEnable` for some reason (for example inside `IPoolable.OnSpawn`), the matching unsubscribe must live in the symmetric callback `IPoolable.OnDespawn`, so that `ObjectPool.ReturnToPool` guarantees a clean detach before the object is queued.

(d) Prevention rule

Subscribe and unsubscribe must always be in a paired lifecycle hook that runs every time the object becomes active and inactive. For pooled objects, the two safe pairs are (`OnEnable`, `OnDisable`) and (`IPoolable.OnSpawn`, `IPoolable.OnDespawn`). `Awake` and `OnDestroy` are not safe for pooled objects, because the pool does not destroy them between uses, so `OnDestroy` never runs and the unsubscribe is never called. After this bug, I check every `+=` on a static event in the project and search for the matching `-=` in the same class. If it is missing, it can become a ghost subscriber later.

7. Retrospective

The choice that helped me the most was keeping GameSetup small. GameSetup only builds the weapon chain and starts WaveManager. GameManager is also small. It only listens to events and shows the win or lose panel. No other class has a direct reference to GameManager. Because of this, I could change the panels, the HUD, or the weapon chain without changing other classes. The codebase stayed easy to read at the end of the project.

If I had more time, I would add an IPoolService class. It would hold the pools and return them by name. My current code uses Inspector references, which works for one scene but is harder to use across many scenes. I would also add a Composite for enemy groups on top of WaveManager. The brief lists Composite as an option, and it would fit well next to the IMovementStrategy I already have.

Repository

<https://github.com/AlpoTheo/CENG454-HW3-AlpDoruk-Sengun-230444401>